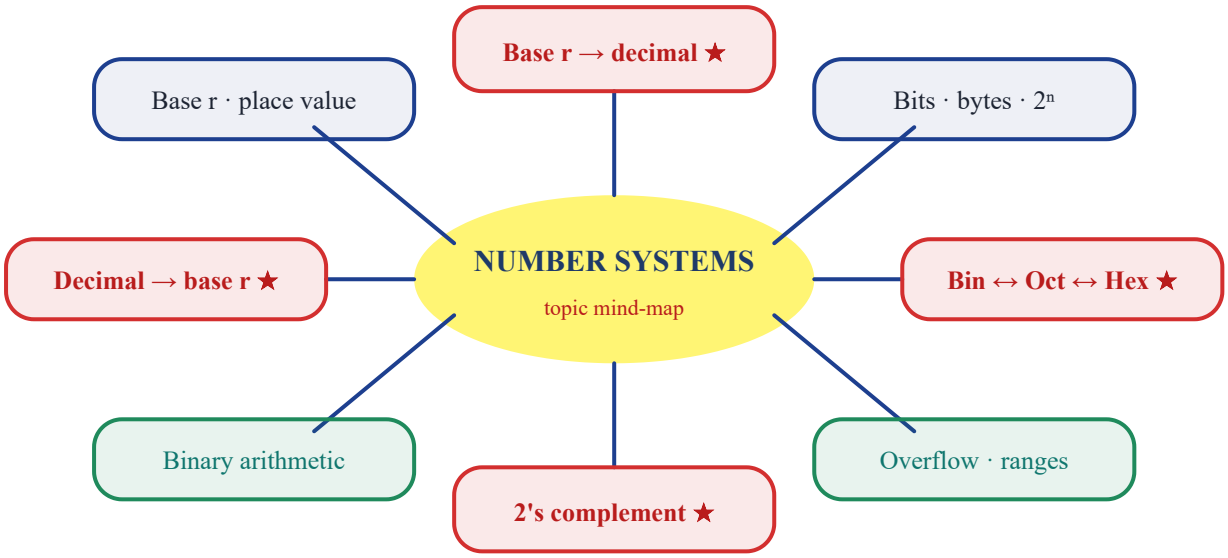
Number Systems

Binary · Octal · Hex · conversions · arithmetic · 2's complement: exam-ready in 11 sheets



↑ the whole topic on one page: red = high-yield, green = method / applied

Syllabus coverage

"Introduction to Computer Systems: … Basic Information Technology, The Internet, **Number system: binary, octal, hexadecimal, binary arithmetic**". This notebook covers positional systems, every base conversion (integer and fraction), arithmetic in bases 2, 8 and 16, complements, ranges and overflow. Codes (BCD, Gray, Excess-3) and signed-number circuits are covered in depth in the DLD Number Systems notes.

High-yield list

① powers of 2 up to 2^{16}
② fraction conversion
③ 3-bit / 4-bit grouping
④ hex / octal sums
⑤ 2's complement & ranges
⑥ overflow rule

Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

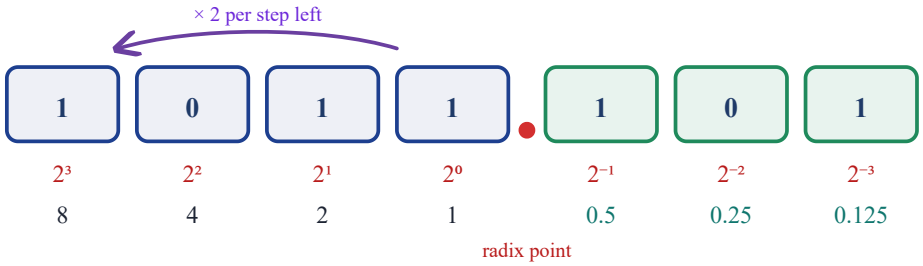
Contents

✓ Positional systems, the map …… p. 2
✓ Bits, 2^n, any base → decimal … p. 3
✓ Decimal → any base …… p. 4
✓ Binary ↔ octal ↔ hex …… p. 5
✓ Binary add & subtract …… p. 6
✓ Multiply, divide, octal/hex sums … p. 7
✓ Complements & signed numbers … p. 8
✓ Complement subtraction, overflow … p. 9
✓ Night-before sheet …… p. 10
✓ Practice MCQs + written …… p. 11

1 Positional Number Systems

Definition: A number system represents quantities with a fixed set of digits and a **base (radix) r** = the number of distinct digits, **including 0**. Digits run from **0 to r − 1**. In a **positional** system each digit's value = digit × $r^{position}$.

Value: $N = d_{n-1}r^{n-1} + \ldots + d_1 r + d_0 + d_{-1}r^{-1} + d_{-2}r^{-2} + \ldots$ (d = digit, r = base)



$(1011.101)_2 = 8 + 2 + 1 + 0.5 + 0.125 = 11.625$: every place is worth r times the place to its right

System	Base	Digits	Largest digit	Example	Other notations
Binary	2	0, 1	1	$(1011)_2$	1011B · 0b1011
Octal	8	0 – 7	7	$(157)_8$	157Q · 0157 (C) · 0o157
Decimal	10	0 – 9	9	$(245)_{10}$	245D
Hexadecimal	16	0 – 9, A – F	F = 15	$(2F)_{16}$	2FH · 0x2F

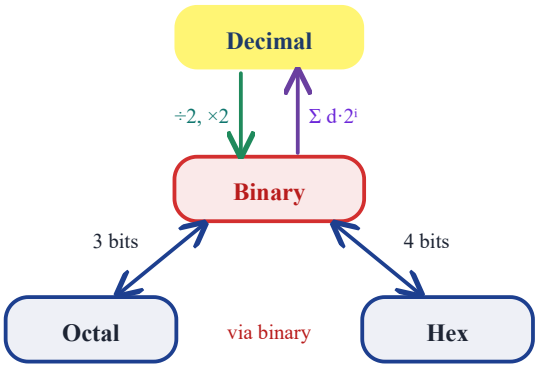
Hex letters: **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**

Why binary?: Electronic devices have **two stable states** (on/off, high/low voltage, magnetised or not). Two levels are easy to make, hard to confuse and map directly onto Boolean logic.

Exam trap: $(10)_r = r$ in every base: $10_2 = 2$, $10_8 = 8$, $10_{16} = 16$. Roman numerals are **non-positional** and have no zero.

Memory trick: "**Base = how many digits; biggest digit = base − 1.**" So 8 or 9 never appears in octal and 2 never appears in binary: $(189)_8$ is invalid.

The map on the right is the whole lesson: sheets 3 – 5 walk every arrow, with integer and fraction parts.



the conversion map

2 Bits, Bytes & Powers of Two

- **Bit** = one binary digit (0/1)
- **Nibble** = 4 bits = **one hex digit**
- **Byte** = 8 bits = two hex digits
- **Word** = the CPU's natural size (16/32/64 bits)
- n bits → **2^n** patterns, unsigned 0 … $2^n - 1$
- n address lines → 2^n locations: 16 lines × 8-bit = 64 KB

n	2^n	n	2^n	n	2^n
0	1	6	64	12	4096
1	2	7	128	13	8192
2	4	8	**256**	14	16384
3	8	9	512	15	32768
4	16	10	**1024**	16	**65536**
5	32	11	2048	20	1048576

Dec	Bin	Oct	Hex	Dec	Bin	Oct	Hex
0	0000	0	0	8	1000	10	8
1	0001	1	1	9	1001	11	9
2	0010	2	2	10	1010	12	A
3	0011	3	3	11	1011	13	B
4	0100	4	4	12	1100	14	C
5	0101	5	5	13	1101	15	D
6	0110	6	6	14	1110	16	E
7	0111	7	7	15	1111	17	F

2^{10} = 1 K, 2^{20} = 1 M, 2^{30} = 1 G, 2^{40} = 1 T. Powers of 8: 1, 8, 64, 512, 4096. Powers of 16: 1, 16, 256, 4096, 65536.

Exam trap: Memory: 1 KB = **1024** bytes (IEC KiB). SI / networking: 1 kB = 1000 bytes, 1 kbps = 1000 bit/s.

3 Any Base → Decimal

Method: Write the **place value** under each digit (r^0, r^1, … to the left; r^{-1}, r^{-2}, … to the right of the point), multiply and add.

Worked examples ✍️:

1. $(1011.101)_2 = 8 + 0 + 2 + 1 + 0.5 + 0 + 0.125 =$ **11.625**
2. $(157.4)_8 = 1\times64 + 5\times8 + 7\times1 + 4\times0.125 = 64 + 40 + 7 + 0.5 =$ **111.5**
3. $(3A7)_{16} = 3\times256 + 10\times16 + 7 = 768 + 160 + 7 =$ **935**
4. $(2F.8)_{16} = 2\times16 + 15 + 8/16 =$ **47.5** · $(212)_3 = 2\times9 + 1\times3 + 2 =$ **23**

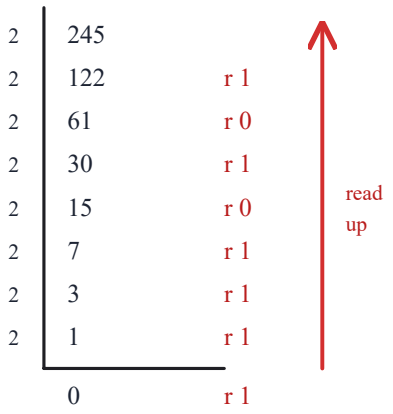
Memory trick: Horner: "**Double and add**" left to right. $(11001)_2$: 1 → 3 → 6 → 12 → **25**. For base r: multiply by r, add the next digit.

Speed check: A binary number ending in **0 is even**; the leftmost 1 tells the size: $1xxxx_2$ lies in 16 … 31.

4 Decimal → Any Base

Integer part: **Divide** by r repeatedly; write each remainder; stop at quotient 0. Read remainders **bottom → top** (first remainder = LSD).

Fraction part: **Multiply** by r repeatedly; take the integer part each time; stop at 0 (or enough digits). Read **top → bottom** (first = MSD).



$245 = (11110101)_2$

Integer examples ✍️:

1. 245 ÷ 8: 30 r **5**, 3 r **6**, 0 r **3** → $\mathbf{(365)_8}$
2. 245 ÷ 16: 15 r **5**, 0 r **15 = F** → $\mathbf{(F5)_{16}}$
3. 1000 ÷ 16: 62 r **8**, 3 r **E**, 0 r **3** → $\mathbf{(3E8)_{16}}$
4. 25 → $(11001)_2$ · check: 16 + 8 + 1 = 25

Shortcut: For small numbers, **subtract the largest power of 2**: 100 = 64 + 32 + 4 → $(1100100)_2$.

Fraction examples ✍️:

0.625 × 2	digit	0.375 × 8	digit	0.375 × 16	digit
1.25	**1**	3.0	**3**	6.0	**6**
0.25 × 2 = 0.5	**0**	stop (fraction 0)		stop	
0.5 × 2 = 1.0	**1**				

$0.625 = (0.101)_2$ · so $\mathbf{25.375 = (11001.011)_2 = (31.3)_8 = (19.6)_{16}}$

Exam trap: $(0.1)_{10} = (0.\mathbf{0011}\ 0011\ 0011\ \ldots)_2$ **never terminates**. A fraction terminates in binary only if its denominator is a power of 2 (0.5, 0.25, 0.375 …). That is why `0.1 + 0.2 != 0.3` in floating point.

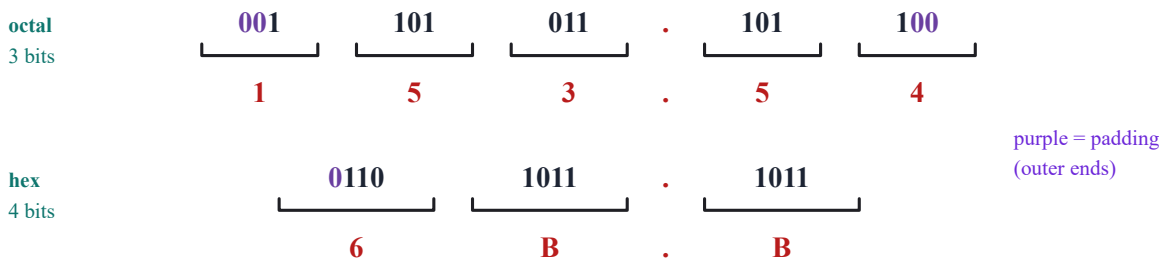
Memory trick: "**Divide → read Up, Multiply → read Down**" (D-U, M-D). Integers grow from the point leftward, fractions grow from the point rightward.

Base r → base s ✍️: Go **through decimal** (or through binary when both are powers of 2): $(212)_3$ → 23 → $\mathbf{(10111)_2}$; $(0.6875)_{10} \times 8 \rightarrow 5.5, 4.0 \rightarrow \mathbf{(0.54)_8}$.

Precision: k fraction bits resolve steps of 2^{-k}. 0.1 cut to 8 bits = $(0.00011001)_2 = 0.09765625$, so stored values are **rounded**.

5 Binary ↔ Octal ↔ Hexadecimal

Rule: $8 = 2^3$ and $16 = 2^4$, so **one octal digit = 3 bits** and **one hex digit = 4 bits**. Group **from the radix point outwards**: integer bits leftwards, fraction bits rightwards; pad with 0s at the **outer** ends only.



$(1101011.1011)_2 = (153.54)_8 = (6B.B)_{16} = 107.6875$

Worked examples ✍️:

1. $(10110001)_2$: 10|110|001 = **$(261)_8$**; 1011|0001 = **$(B1)_{16}$**
2. $(157)_8 \rightarrow$ 001 101 111 → 0110 1111 = **$(6F)_{16}$** (= 111)
3. $(3FA)_{16} \rightarrow$ 0011 1111 1010 → 001 111 111 010 = **$(1772)_8$** (= 1018)
4. $(A5)_{16}$ = 1010 0101 = $(245)_8$ = 165

Exam trap: Fraction bits are padded on the **right**: .1011 → .101 100 = $.54_8$, not .001 011 = $.13_8$.

Octal ↔ hex has no direct grouping: go **through binary**.

Memory trick: "**Oct-3, Hex-4**": the exponent of 2 is the group size. Base 4 = 2^2 → groups of 2; base 32 = 2^5 → groups of 5.

Digit counts: an n-bit number needs **⌈n/3⌉ octal** and **⌈n/4⌉ hex** digits: 32 bits = 8 hex digits, 11 octal digits.

6 Where Each Base Is Used

Base	Real use	Example
Binary	machine code, bit masks, flags	x & 0b1111 keeps the low nibble
Octal	Unix file permissions (3 bits rwx)	chmod **755** = rwx r-x r-x
Decimal	IPv4 dotted decimal (4 octets)	192.168.1.1, each part 0 – 255
Hexadecimal	addresses, MAC, IPv6, colours, Unicode	0x7FFF · #FF8800 · U+0985

7 Binary Addition

A + B (+ carry)	Sum	Carry
0 + 0	0	0
0 + 1 = 1 + 0	1	0
1 + 1 = $(10)_2$	**0**	**1**
1 + 1 + 1 = $(11)_2$	**1**	**1**

	1	1	1	1	carries
		1	**0**	**1**	**1** = 11
+		**1**	**1**	**0**	**1** = 13
	1	**1**	**0**	**0**	**0** = 24 ✓

1011 + 1101 = 11000

More examples ✍️:

1. $(101)_2 + (11)_2 = \mathbf{(1000)_2}$ (5 + 3 = 8)
2. $(101.11)_2 + (11.01)_2 = \mathbf{(1001.00)_2}$ (5.75 + 3.25 = 9): line up the points first
3. Three rows: 111 + 101 + 11 = $\mathbf{(1111)_2}$ (7 + 5 + 3 = 15): a column sum of 3 = $(11)_2$ writes 1, carries 1

Exam trap: Boolean OR: 1 + 1 = **1**. Binary arithmetic: 1 + 1 = **10**. Same "+" sign, different meaning: read the question.

8 Binary Subtraction (Borrow Method)

A − B	Diff	Borrow
0 − 0	0	0
1 − 0	1	0
1 − 1	0	0
0 − 1 = 10 − 1	**1**	**1**

Worked example ✍️: $(11010)_2 - (01111)_2$

bit 0: 0 − 1 → borrow → 10 − 1 = 1

bit 1: (1 − 1) − 1 → borrow → 1

bit 2: (0 − 1) − 1 → borrow → 0

bit 3: (1 − 1) − 1 → borrow → 1

bit 4: 1 − 1 − 0 = 0 → $\mathbf{(01011)_2}$ = 11 ✓ (26 − 15)

More: $(1010)_2 - (11)_2 = \mathbf{(111)_2}$ (10 − 3 = 7) · $(1101.1)_2 - (110.11)_2 = \mathbf{(110.11)_2}$ (13.5 − 6.75 = 6.75).

Memory trick: A borrow brings "**one base**" into the column: 2 in binary, 8 in octal, 16 in hex. Check every answer: **difference + subtrahend = minuend**.

9 Binary Multiplication & Division

× rules = AND: 0×0 = 0×1 = 1×0 = 0, 1×1 = 1. Method: **shift and add**.

÷ = long division; at each step the divisor "goes" **once or not at all**.

```
    1 1 0 1   (13)
  × 1 0 1     (5)
-------------
    1 1 0 1   × 1
  0 0 0 0     × 0, shift 1
1 1 0 1       × 1, shift 2
-------------
1 0 0 0 0 0 1 (65)
```

partial products, then add

```
          1 0 0 1   (54 ÷ 6 = 9)
      ┌──────────
110   │ 1 1 0 1 1 0
    − 1 1 0
      ---------
        0 1 1 0     01, 011 < 110
                    → two 0s in q
      − 1 1 0
      ---------
            0       remainder 0
```

$(110110)_2 \div (110)_2 = \mathbf{(1001)_2}$ · 100111 ÷ 101 = **111 r 100** (39 ÷ 5 = 7 r 4)

Shift rule: Shift left k places = × $\mathbf{2^k}$; shift right = ÷ $\mathbf{2^k}$ (integer part). 1011 << 2 = 101100 (11 → 44).

Exam trap: $(1101)_2 \times (101)_2$ is **1000001**, not 1000011. Convert to decimal and check (13 × 5 = 65 = 64 + 1).

10 Octal & Hexadecimal Arithmetic

Rule: Add column by column in decimal. If the sum ≥ base, **write sum − base, carry 1**. When subtracting, a borrow adds **8** (octal) or **16** (hex) to the column.

Octal ✍️:

1. 457 + 365: 7 + 5 = 12 = 8 + **4** c1 · 5 + 6 + 1 = 12 → **4** c1 · 4 + 3 + 1 = 8 → **0** c1 → $\mathbf{(1044)_8}$ (303 + 245 = 548)
2. 712 − 345: 2 − 5 → 10 − 5 = **5** · 0 − 4 → 8 − 4 = **4** · 6 − 3 = **3** → $\mathbf{(345)_8}$
3. $(64)_8 \times 7$: 4×7 = 28 = 3×8 + **4** · 6×7 + 3 = 45 = 5×8 + **5** → $\mathbf{(554)_8}$

Hex ✍️:

1. 9A + 7C: A + C = 22 = 16 + **6** c1 · 9 + 7 + 1 = 17 → **1** c1 → $\mathbf{(116)_{16}}$ (154 + 124 = 278)
2. 3A2 − 1F7: 2 − 7 → 18 − 7 = **B** · 9 − F → 25 − 15 = **A** · 2 − 1 = **1** → $\mathbf{(1AB)_{16}}$ (930 − 503 = 427)
3. $(2A)_{16} \times 3 = 42 \times 3 = 126 = \mathbf{(7E)_{16}}$

Memory trick: Think in decimal, write in the base: "**over 7? take 8 away; over 15? take 16 away**", and carry 1.

Exam trap: $(FF)_{16} + 1 = \mathbf{(100)_{16}} = 256$, and $(77)_8 + 1 = (100)_8 = 64$. "10" in hex is sixteen.

11 r's and (r − 1)'s Complement

Definition: For an n-digit number N in base r: **(r − 1)'s complement = $(r^n - 1) - N$** (subtract every digit from r − 1); **r's complement = $r^n - N$** = (r − 1)'s complement + 1. Binary: **1's** and **2's**; decimal: **9's** and **10's**.

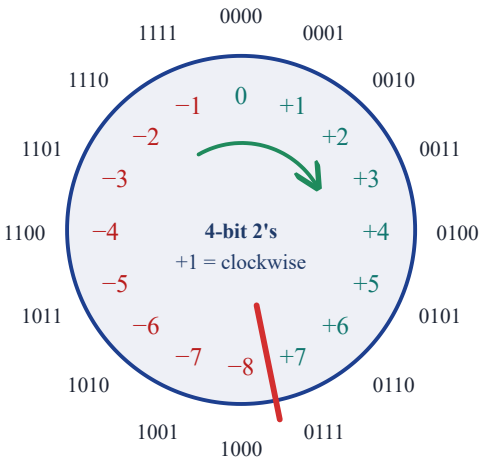
N (8-bit)	1's (flip)	2's (+1)
01011000	10100111	**10101000**
00101101 (45)	11010010	11010011
00000101 (5)	11111010	11111011
00000000	11111111	00000000
decimal 0725	9274 (9's)	9275 (10's)

Memory trick: 2's shortcut: From the right, **copy every bit up to and including the first 1**, then flip the rest. 0101|1000: copy "1000", flip "0101" →**1010 1000**.

12 Signed Numbers & Ranges

MSB = sign bit (0 = +, 1 = −). Three ways to write **−5** in 8 bits (details of the circuits: DLD notes):

Representation	−5 (8-bit)	Range for n bits	8-bit range	Zeros
Unsigned	(not possible)	$0 \dots 2^n - 1$	0 … 255	1
Sign-magnitude	10000101	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127	**2**
1's complement	11111010	$-(2^{n-1} - 1) \dots +(2^{n-1} - 1)$	−127 … +127	**2**
2's complement	11111011	$\mathbf{-2^{n-1} \dots +(2^{n-1} - 1)}$	**−128 … +127**	1



red bar: +7 → −8 is where overflow happens

MSB weight: In 2's complement the MSB weighs $\mathbf{-2^{n-1}}$. 11111011 = −128 + 64 + 32 + 16 + 8 + 2 + 1 = **−5**.

Exam trap: 8-bit 2's complement holds **−128** but not +128; the 2's complement of 10000000 is 10000000 itself. Sign-magnitude and 1's complement waste a pattern on **−0**.

16-bit: unsigned 0 … 65535, 2's complement **−32768 … +32767**.

13 Subtraction by Complements

2's complement method: A − B = **A + (2's complement of B)**. **End carry → discard it**, the result is positive. **No end carry** → the result is negative; take its 2's complement for the magnitude.

25 − 13 (8-bit) ✍️:
25 = 00011001, −13 = 11110011
00011001 + 11110011 = **1** 00001100
carry out → discard → **00001100 = +12**

13 − 25 (8-bit) ✍️:
13 = 00001101, −25 = 11100111
00001101 + 11100111 = 11110100 (no carry)
negative → 2's comp = 00001100 → **−12**

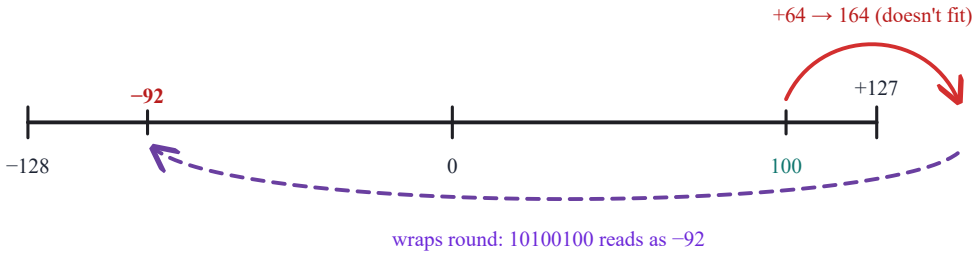
1's complement method ✍️: 1011 − 0110 (11 − 6): 1011 + 1001 = **1** 0100 → **end-around carry**: 0100 + 1 = **0101** = 5. No carry → negative, answer = −(1's complement of the sum).

10's complement ✍️: 725 − 318 = 725 + 682 = **1**407 → drop the carry → **407**. (682 = 10's complement of 318.)

Why computers use 2's complement: one adder does both + and − · only **one zero** · no end-around carry · the sign falls out automatically.

14 Overflow

Rule: Overflow = the true result is outside the n-bit range. In 2's complement it happens only when **two numbers of the same sign** give a result of the **opposite sign**; equivalently **carry into MSB ≠ carry out of MSB**. Adding numbers of opposite signs **never** overflows.



Example ✍️: 100 + 64: 01100100 + 01000000 = **10100100**: two positives gave a negative (−92) → **overflow**.

Example ✍️: (−100) + (−64): 10011100 + 11000000 = 1 **01011100** (+92): two negatives gave a positive → **overflow**.

Exam trap: **Carry ≠ overflow.** For unsigned numbers the carry-out of the MSB signals overflow; for 2's complement a carry-out is normal (25 − 13 above) and is simply discarded.

Night-Before-the-Exam Sheet

Fact sheet:

Base r: digits 0 … r − 1 · $(10)_r = r$

Dec → r: ÷ r read up · fraction × r read down

$2^8 = 256 \cdot 2^{10} = 1024 \cdot 2^{16} = 65536$

1 + 1 = 10 · 1 + 1 + 1 = 11 · 0 − 1 = 1 b1

1's = flip · 2's = flip + 1

2's range: $-2^{n-1} \ldots 2^{n-1} - 1$

A − B = A + 2's(B); carry → discard

$N = \Sigma\, d_i \times r^i$ (i < 0 after the point)

Octal = 3 bits · Hex = 4 bits (from the point)

n bits → 2^n values, unsigned max $2^n - 1$

Octal carry at 8 · hex carry at 16

r's comp = $r^n - N$ · (r−1)'s = $r^n - 1 - N$

S-M / 1's: $\pm(2^{n-1} - 1)$, two zeros

Overflow: $C_{in}(MSB) \neq C_{out}(MSB)$

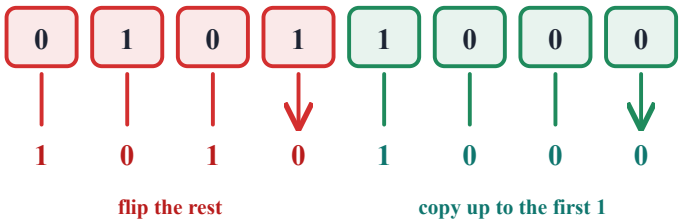
Top 12 one-liners for MCQs

1. $(1011)_2 = 11 \cdot (11001)_2 = 25$
2. $245 = 11110101 = 365_8 = F5_{16}$
3. $1000 = 3E8_{16} = 1750_8$
4. $157_8 = 111 = 6F_{16}$
5. $0.625 = 0.101_2$
6. 0.1 is non-terminating in binary
7. $9A + 7C = 116_{16}$
8. 1101 × 101 = 1000001
9. 2's(01011000) = 10101000
10. 8-bit 2's: −128 … +127
11. chmod 755 = rwxr-xr-x
12. 32 bits = 8 hex digits

Top traps: Boolean 1 + 1 = 1, binary 1 + 1 = 10 · remainders read bottom-up, fraction digits top-down · pad fraction groups on the right · octal has no 8 or 9 · $(FF)_{16} + 1 = 100_{16}$ · −128 fits in 8 bits, +128 does not · carry ≠ overflow · 1 KB = 1024 B

All tricks together: Base = number of digits, largest = base − 1 · Horner: double and add · Divide → read Up, Multiply → read Down · Oct-3, Hex-4 · a borrow brings one base · over 7 take 8, over 15 take 16 · 2's: copy up to the first 1, flip the rest · MSB weighs -2^{n-1}

Written-exam tips: Show the division ladder and the multiplication column; write the base as a subscript on **every** number; finish with a decimal check line ("Check: 3×64 + 6×8 + 5 = 245").



the 2's complement shortcut, read from the right

✓ Practice MCQs (exam pattern, 10 marks)

1. $(1101.01)_2$ in decimal is:

(a) 13.25 (b) 13.5
(c) 11.25 (d) 12.25

2. $(3A)_{16}$ in octal is:

(a) 70 (b) 72
(c) 74 (d) 62

3. Range of an 8-bit 2's complement number:

(a) 0 … 255 (b) −127 … +127

(c) −128 … +127 (d) −128 … +128

4. The 2's complement of 10110100 is:

(a) 01001011 (b) 01001101

(c) 10110100 (d) 01001100

5. $(0.1)_{10}$ in binary is:

(a) 0.0001 (b) 0.1

(c) 0.000110011… (d) 0.0011

6. $(457)_8 + (365)_8 = ?$

(a) $(1044)_8$ (b) $(822)_8$

(c) $(1034)_8$ (d) $(1144)_8$

7. Which is NOT a valid octal number?

(a) 7770 (b) 1289

(c) 2047 (d) 0

8. How many distinct values can 12 bits hold?

(a) 4095 (b) 2048

(c) 1024 (d) 4096

9. 01100100 + 01000000 in 8-bit 2's complement (OVF = overflow):

(a) 10100100, no OVF (b) 10100100, OVF

(c) 00100100, OVF (d) 00100100, carry

10. $(1000)_{10}$ in hexadecimal is:

(a) 3E8 (b) 3F8

(c) 2E8 (d) 3E0

Answer key → 1-a · 2-b · 3-c · 4-d · 5-c · 6-a · 7-b · 8-d · 9-b · 10-a

Written question: (a) Convert $(156.6875)_{10}$ to binary, octal and hexadecimal. (b) Using 8-bit 2's complement, compute 45 − 72 and say whether overflow occurs. 8 marks

1. 156 ÷ 2 → remainders 0, 0, 1, 1, 1, 0, 0, 1 → read up: **10011100**; 0.6875 × 2 → 1.375, 0.75, 1.5, 1.0 → **.1011** 2
2. $(156.6875)_{10}$ = **$(10011100.1011)_2$** → 010 011 100 . 101 100 = **$(234.54)_8$** → 1001 1100 . 1011 = **$(9C.B)_{16}$** 2
3. 45 = 00101101, 72 = 01001000 → 2's(72) = 10111000; 00101101 + 10111000 = **11100101**, no end carry 2
4. Negative: 2's(11100101) = 00011011 = 27 → answer **−27**. Signs differed, so **no overflow**. Check: 45 − 72 = −27 ✓ 2

Before the exam ✓

- ✓ Can write 2^0 … 2^{16} and 0 – F in binary from memory
- ✓ Can convert integer and fraction parts both ways
- ✓ Can add and subtract in bases 2, 8 and 16
- ✓ Can do 2's complement subtraction and spot overflow